

GABARITO OFICIAL & CRITÉRIOS DE CORREÇÃO

Avaliação N1 — Estrutura de Dados (4º Período) • Ciência da Computação
Docente: Prof. Me. Karan Luciano | Semestre Letivo 2026.1 | Valor Total da Avaliação: 30,0 Pontos

1. GABARITO DAS QUESTÕES OBJETIVAS (Q01 A Q10 — TOTAL: 15,0 PONTOS)

Questão	A	B	C	D	E	Conteúdo Avaliado	Gabarito	Valor
01	X					Conceito de Ponteiro e Gestão de Memória RAM (Guardar Endereço de Memória Física)	ALTERNATIVA A	1,5 pt
02		X				Operadores de Indireção (*ptr = 25 altera diretamente a variável para 25)	ALTERNATIVA B	1,5 pt
03		X				Aritmética de Ponteiros (*(v + 3) avança 12 bytes → 0x100C e valor 40)	ALTERNATIVA B	1,5 pt
04	X					Alocação Dinâmica na Heap com função padrão malloc() e operador sizeof	ALTERNATIVA A	1,5 pt
05		X				Ciclo de Vida da Memória e Caracterização Formal de Vazamento (Memory Leak)	ALTERNATIVA B	1,5 pt
06		X				Registros Dinâmicos e Sintaxe ANSI C (ponteiro_no->valor = 100;)	ALTERNATIVA B	1,5 pt
07		X				Pré-condição Mandatória da Busca Binária (Organização Monotônica / Ordenada)	ALTERNATIVA B	1,5 pt
08	X					Análise Assintótica: Busca Binária $O(\log_2 n)$ em 1.000.000 de registros (≤ 20 comparações)	ALTERNATIVA A	1,5 pt
09		X				Mecânica do Bubble Sort (Varredura de Pares Adjacentes e Pior Caso Quadrático $O(n^2)$)	ALTERNATIVA B	1,5 pt
10		X				Algoritmos Avançados: Merge Sort e Paradigma de Divisão e Conquista	ALTERNATIVA B	1,5 pt
Subtotal das Questões Objetivas (10 itens × 1,5 pt):								15,0 pts

2. SÍNTESE DAS QUESTÕES DISCURSIVAS E STATUS DE ANULAÇÃO (TOTAL: 15,0 PONTOS)

Questão	Tema / Objeto de Avaliação	Valor	Status	Critério de Atribuição da Nota
QUESTÃO 11	Ponteiros, Passagem por Referência e Escopo (Rotina misterio)	7,50 pts	RESOLVIDA	Item a: 3,75 pts Item b: 3,75 pts
QUESTÃO 12	Encadeamento Dinâmico e Ciclo de Vida na Heap (Nós Dinâmicos)	—	ANULADA	Pontuação redistribuída para Q11 e Q13
QUESTÃO 13	Paradigmas de Busca e Análise de Complexidade (500.000 Pacotes)	7,50 pts	RESOLVIDA	Item a: 3,75 pts Item b: 3,75 pts
QUESTÃO 14	Ordenação e Escalabilidade de Algoritmos (Bubble Sort vs Merge Sort)	—	ANULADA	Pontuação redistribuída para Q11 e Q13
Subtotal das Questões Discursivas (Q11: 7,50 pts + Q13: 7,50 pts):		15,0 pts	15,0 Pontos Redistribuídos nas Questões Válidas	
PONTUAÇÃO TOTAL DA AVALIAÇÃO N1 (Objetivas: 15,0 + Discursivas: 15,0):		30,0 pts	Escala Regimental Máxima: 30,0 Pontos	

3. JUSTIFICATIVAS PEDAGÓGICAS — QUESTÕES OBJETIVAS (1,5 PT CADA)

Questão 01 — Alternativa A Ponteiros e Memória (1,5 pt)

Um ponteiro é uma variável cujo objetivo primordial é **guardar o endereço de memória física** onde a variável referenciada reside, viabilizando manipulação indireta e eficiente da memória RAM.

Questão 02 — Alternativa B Indireção de Ponteiros (1,5 pt)

O operador de desreferenciação (`*ptr = 25;`) acessa e sobrescreve diretamente a posição original de memória de `numero`. Assim, `printf("%d", numero)` imprime **25**.

Questão 03 — Alternativa B Aritmética de Ponteiros (1,5 pt)

Com `sizeof(int) = 4 bytes`, a expressão `*(v + 3)` avança $3 \times 4 = 12$ bytes (`0x0C` em hexadecimal). Partindo de `0x1000`, acessa-se **0x100C**, recuperando o quarto elemento (`v[3] = 40`).

Questão 04 — Alternativa A Alocação Dinâmica (1,5 pt)

A função padrão da biblioteca `<stdlib.h>` para alocação sob demanda de blocos contíguos na *Heap* é `malloc()`, na qual a quantidade necessária de bytes é informada via operador `sizeof`.

Questão 05 — Alternativa B Ciclo de Vida da Memória (1,5 pt)

Blocos alocados na *Heap* com `malloc()` sem chamada correspondente a `free()` tornam-se inacessíveis quando seus ponteiros são destruídos, caracterizando **Vazamento de Memória (*Memory Leak*)**.

Questão 06 — Alternativa B Registros e Nós Dinâmicos (1,5 pt)

Em ANSI C, o acesso a campos de uma estrutura via ponteiro adota o operador seta: `ponteiro_no->valor = 100;`, sendo a forma canônica equivalente a `(*ponteiro_no).valor = 100;`.

Questão 07 — Alternativa B Pré-condição Busca Binária (1,5 pt)

A Busca Binária requer obrigatoriamente que a estrutura esteja organizada em **ordem monotônica** (estritamente crescente ou decrescente), permitindo o descarte lógico consistente de metades a cada passo.

Questão 08 — Alternativa A Complexidade Logarítmica (1,5 pt)

A Busca Binária possui complexidade **$O(\log_2 n)$** . Para 1.000.000 de registros, são necessárias no máximo $\lceil \log_2 10^6 \rceil = 20$ comparações, contrastando com as até 10^6 da busca linear $O(n)$.

Questão 09 — Alternativa B Mecânica do Bubble Sort (1,5 pt)

O Bubble Sort percorre o vetor comparando pares de elementos vizinhos e realizando permutações sucessivas, exibindo complexidade de tempo de pior caso quadrática **$O(n^2)$** .

Questão 10 — Alternativa B Merge Sort e Divisão/Conquista (1,5 pt)

O Merge Sort fundamenta-se no paradigma de **Divisão e Conquista**: divide o vetor recursivamente até partições unitárias e recombina-as ordenadamente por intercalação com tempo garantido **$O(n \log n)$** .

QUESTÃO 11 — ANÁLISE DE CÓDIGO: PONTEIROS, PASSAGEM POR REFERÊNCIA E ESCOPO

VALOR: 7,50 PTS

Código C da Questão:

```
#include <stdio.h>
void misterio(int *a, int *b) { int temp = *a; *a = *b; *b = temp + 5; }
int main() {
    int v[4] = {10, 20, 30, 40}; int *p1 = &v[1]; int *p2 = &v[3];
    misterio(p1, p2);
    for (int i = 0; i < 4; i++) { printf("%d ", v[i]); }
    printf(" "); return 0;
}
```

Item a) Saída Exata Impressa e Teste de Mesa (3,75 pts)

Saída Exata no Console:

10 40 30 25

Teste de Mesa e Rastreamento de Execução na Memória:

Etapas / Instrução	v[0]	v[1]	v[2]	v[3]	Ponteiros e Variáveis Locais
Inicialização em main	10	20	30	40	p1 = &v[1] (aponta p/ 20); p2 = &v[3] (aponta p/ 40)
temp = *a;	10	20	30	40	temp = 20 (lê o conteúdo de v[1] através de *a)
*a = *b;	10	40	30	40	Célula original v[1] recebe 40 (conteúdo de v[3])
*b = temp + 5;	10	40	30	25	Célula original v[3] recebe 20 + 5 = 25
Laço for de saída	10	40	30	25	Impressão sequencial: 10 40 30 25

Barema de Correção do Item (a) — Total: 3,75 pts:

- Sequência de valores correta no console (10 40 30 25): **1,75 pt.**
- Rastreamento lógico / teste de mesa detalhado demonstrando os valores dos ponteiros e posições: **2,00 pts.**

Item b) Fundamentação Técnica: Passagem por Referência vs. Valor (3,75 pts)

Fundamentação Técnica Esperada:

- Mecanismo de Passagem por Referência Simulada:** Em linguagem C, toda passagem é formalmente por valor (cópia). Contudo, ao passarmos os ponteiros p1 e p2 (endereços físicos de v[1] e v[3]), os parâmetros formais a e b passam a apontar diretamente para as células de memória alocadas na pilha (Stack) da main.
- Atuação do Operador de Indireção (*):** As atribuições *a = ... e *b = ... não modificam variáveis locais da função, mas sim as células originais do vetor na memória física. Logo, as alterações persistem após o término do frame de misterio.
- Contraste com Passagem por Valor Tradicional:** Se a assinatura fosse void misterio(int a, int b) e a chamada misterio(v[1], v[3]), seriam criadas variáveis locais contendo cópias escalares dos números 20 e 40 na pilha da função. Quaisquer alterações em a e b seriam descartadas no return, e o vetor v na main permaneceria inalterado com {10, 20, 30, 40}.

Barema de Correção do Item (b) — Total: 3,75 pts:

- Explicação do acesso direto à memória da main via desreferenciação (*): **1,85 pt.**
- Diferenciação precisa e rigorosa frente à passagem tradicional por valor (cópia descartável): **1,90 pt.**

COMUNICADO OFICIAL DE ANULAÇÃO:

Esta questão foi **ANULADA** pela banca docente. A pontuação correspondente foi **redistribuída para as duas questões discursivas válidas da avaliação (Q11 e Q13)**, que passam a valer **7,50 pontos cada** (totalizando os 15,0 pontos do bloco discursivo). Nenhum estudante é prejudicado.

Gabarito de Referência & Resolução Pedagógica (Para Fins de Estudo):**Resolução do Item a:**

- **Saída impressa no console:** [10] -> [20] -> NULL.
- **Conexão lógica dos nós:** n1 e n2 residem na pilha (*Stack*) e apontam para blocos alocados na *Heap*. A instrução n1->next = n2; armazena o endereço do segundo nó dentro do ponteiro de encadeamento do primeiro nó. O laço percorre sequencialmente a partir de n1 até encontrar p == NULL.

Resolução do Item b:

- **Patologia Ocorrida:** Ocorre um **Vazamento de Memória (*Memory Leak*)** no nó n2. Ao executar free(n1); seguido de n2 = NULL; , o programador aterrou o ponteiro sem liberar a memória na *Heap* alocada para n2. Esse bloco permanece reservado e inacessível pelo restante da execução.
- **Correção em Código C:** Inserir a desalocação do nó n2 antes de anular os ponteiros:

```
free(n1);
free(n2); // Liberação mandante do segundo nó dinâmico
n1 = NULL;
n2 = NULL;
```

QUESTÃO 13 — DISCURSIVA: PARADIGMAS DE BUSCA E ANÁLISE DE COMPLEXIDADE

VALOR: 7,50 PTS

Item a) Preenchimento do Quadro Comparativo Analítico (3,75 pts)

Critério de Análise	Busca Sequencial (Linear)	Busca Binária
Pré-requisito dos dados	Nenhum pré-requisito. Opera com dados em qualquer ordem, desordenados ou arbitrários.	Ordenação prévia mandatória em ordem monotônica crescente ou decrescente.
Complexidade (Pior Caso)	$O(n)$ — linear. Para 500.000 pacotes, realiza até 500.000 comparações .	$O(\log_2 n)$ — logarítmica. Para 500.000 pacotes, realiza no máximo 19 comparações ($\lceil \log_2 500.000 \rceil = 19$).
Cenário de uso recomendado	Coleções pequenas ou bases altamente dinâmicas com frequentes inserções e raras buscas.	Bases volumosas estáveis ou com alta frequência de consultas repetidas, amortizando o custo de ordenação prévia.

Barema de Correção do Item (a) — Total: 3,75 pts:

- Pré-requisitos dos dados corretos: **1,25 pt.**
- Complexidade Big-O e quantidade de comparações no pior caso (500.000 vs. 19): **1,25 pt.**
- Cenários recomendados coerentes e tecnicamente fundamentados: **1,25 pt.**

Item b) Custo de Reordenação e Viabilidade Econômica/Computacional (3,75 pts)

Fundamentação Técnica Esperada:

- **Impacto das Inserções no Vetor:** Inserir novas encomendas no fim de um vetor desordenado custa apenas $O(1)$. Contudo, para manter a Busca Binária apta, cada novo lote de pacotes exigiria ou uma reordenação completa (custo $O(n \log n)$ via QuickSort ou MergeSort) ou uma inserção ordenada por deslocamento de memória (custo $O(n)$).
- **Inviabilidade da Busca Binária sobre Vetor Rígido:** Em uma base de 500.000 registros, reordenar a coleção a cada minuto consome cerca de $500.000 \times 19 \approx 9,5 \times 10^6$ operações por ciclo. O custo computacional de ordenar continuamente sobrecarrega a CPU dos servidores, provocando gargalos e alta latência, o que torna a Busca Binária sobre vetor estático inviável e antieconômica. A economia de 19 passos de busca é completamente sobrepujada pela penalidade contínua da ordenação.
- **Soluções de Engenharia de Software Recomendadas:**
 1. Se o volume de consultas for moderado, a **Busca Sequencial** com inserção instantânea $O(1)$ apresenta custo computacional global inferior.
 2. Se a alta demanda concorrente de clientes no app exigir busca veloz em base dinâmica, deve-se substituir o vetor rígido por estruturas auto-balanceadas (como **Árvores AVL / Rubro-Negras**, que garantem inserção e busca em $O(\log n)$) ou **Tabelas Hash** com custo médio de inserção e consulta em $O(1)$.

Barema de Correção do Item (b) — Total: 3,75 pts:

- Análise comparativa do custo da reordenação ($O(n \log n)$ ou $O(n)$ vs. $O(1)$): **1,85 pt.**
- Diagnóstico correto de inviabilidade e proposta de soluções arquiteturais adequadas: **1,90 pt.**

COMUNICADO OFICIAL DE ANULAÇÃO:

Esta questão foi **ANULADA** pela banca docente. A pontuação correspondente foi **redistribuída para as duas questões discursivas válidas da avaliação (Q11 e Q13)**, que passam a valer **7,50 pontos cada** (totalizando os 15,0 pontos do bloco discursivo). Nenhum estudante é prejudicado.

Gabarito de Referência & Resolução Pedagógica (Para Fins de Estudo):**Resolução do Item a — Análise Assintótica Big-O:**

- Para conjuntos triviais ($n = 20$), tanto $n^2 = 400$ quanto $n \log_2 n \approx 86$ operações são executadas em frações microscópicas de milissegundo.
- Para $n = 1.000.000$ de registros:
 - **Bubble Sort $O(n^2)$** : requer da ordem de $(10^6)^2 / 2 \approx 5 \times 10^{11}$ comparações e trocas. Em processadores modernos, esse volume astronômico de instruções consome horas ou dias, congelando a aplicação e provocando *timeout*.
 - **Merge Sort $O(n \log_2 n)$** : requer cerca de $10^6 \times 20 \approx 2 \times 10^7$ operações, concluídas com alta eficiência em apenas alguns segundos.

Resolução do Item b — Complexidade Espacial e Sistemas Embarcados:

- **Memória Auxiliar $O(n)$ do Merge Sort**: O Merge Sort não é um algoritmo *in-place*. Durante a etapa de combinação ordenada (intercalação / *merge*), ele obrigatoriamente aloca um vetor auxiliar de tamanho proporcional à partição para depois copiar os dados ordenados de volta, consumindo $O(n)$ de memória adicional na RAM.
- **Sistemas Embarcados de Baixo Custo**: Em microcontroladores com severa limitação de RAM (ex.: 2 KB a 32 KB), a sobrecarga de alocar um segundo vetor proporcional a n pode provocar esgotamento de memória e travamento imediato do sistema. Nesses cenários restritos, algoritmos *in-place* com consumo espacial auxiliar constante **$O(1)$** (como Insertion Sort para volumes pequenos ou Heap Sort para conjuntos maiores) são preferíveis ou estritamente mandatórios.